

NextJS

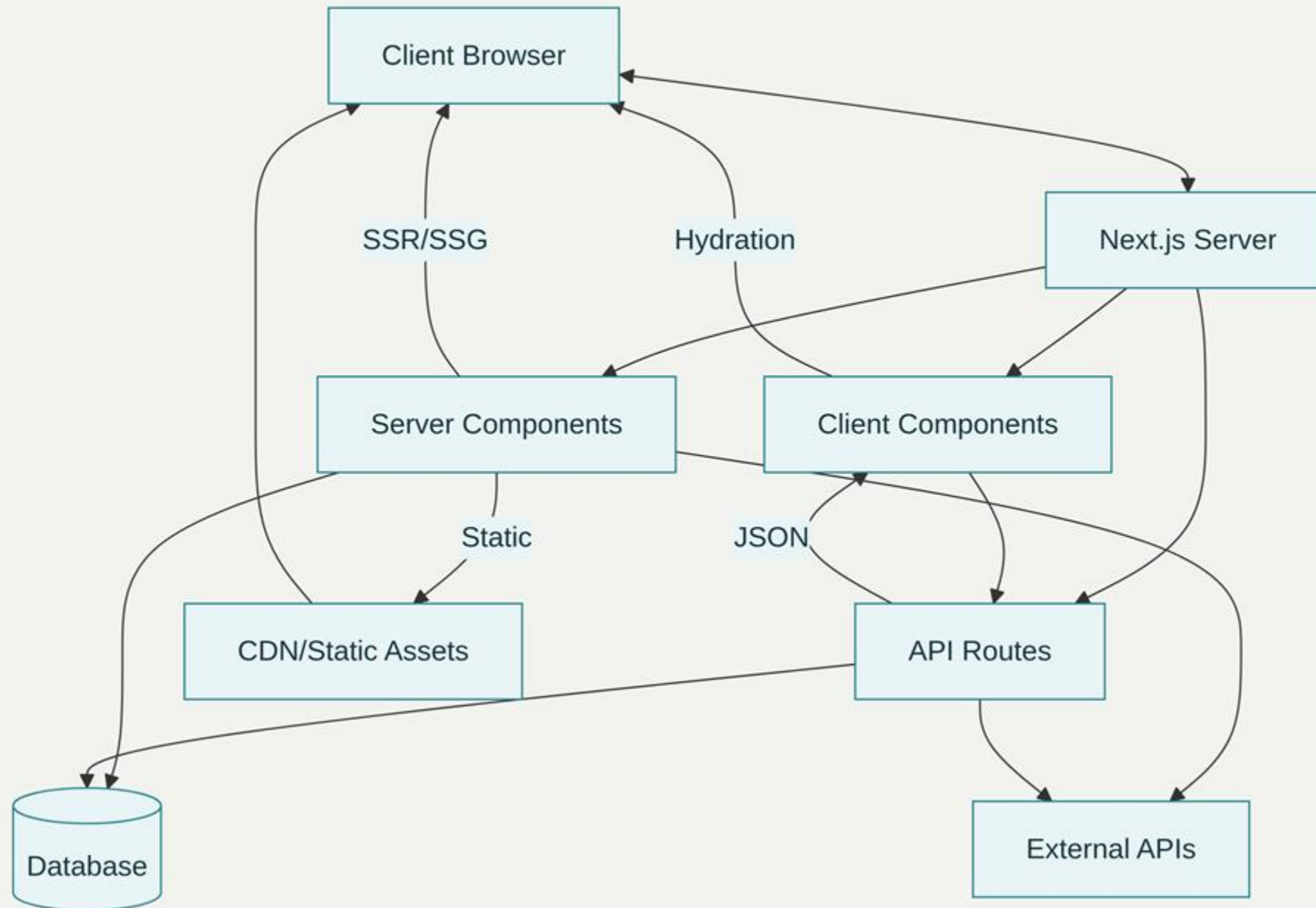
Prof. Lázaro



Fundamentos

- Next.js é um framework full-stack para React mantido pela Vercel, que combina renderização server-side, geração de sites estáticos, rotas de API e otimizações automáticas.
- Desde sua introdução em 2016, tornou-se a escolha preferida para aplicações React em produção, sendo usado por mais de 50% dos desenvolvedores em sessões de desenvolvimento com a versão 15.
- O framework resolve desafios fundamentais do desenvolvimento React tradicional: SEO limitado, performance inicial lenta, configuração complexa de build e falta de estrutura opinativa para aplicações full-stack.
- Com Next.js 15, foram introduzidas melhorias significativas incluindo suporte total ao React 19, Turbopack estável para desenvolvimento e novas APIs de navegação.

Arquitetura e Fluxo de Dados



App Router

Roteamento baseado em pastas com arquivos especiais (page.tsx, layout.tsx, error.tsx)

Server Components por padrão, resultando em bundles JavaScript menores

Layouts aninhados e dinâmicos que permitem interfaces mais sofisticadas

Data fetching integrado com async/await diretamente nos componentes

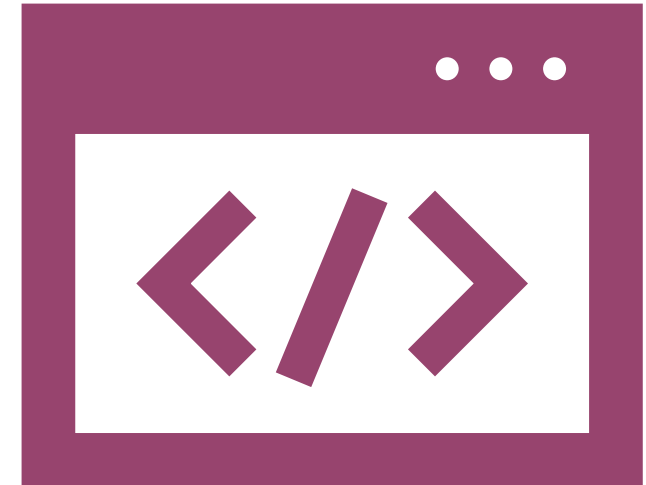
Suporte nativo a streaming e Suspense para carregamento progressivo

Estratégias de renderização

```
// Exemplo de SSG com generateStaticParams
export async function generateStaticParams() {
  const posts = await getPosts()
  return posts.map((post) => ({ slug: post.slug }))
}
```

```
export default async function PostPage({ params }: { params: { slug: string } }) {
  const post = await getPost(params.slug)
  return <PostContent post={post} />
}
```

- Static Site Generation (SSG)
 - SSG representa a estratégia mais performática, onde páginas são pré-renderizadas durante o build time.
 - É ideal para conteúdo que não muda frequentemente como blogs, documentação e sites de marketing.
 - O conteúdo é servido instantaneamente através de CDNs globais, proporcionando experiência de usuário superior.



Server-Side Rendering (SSR)

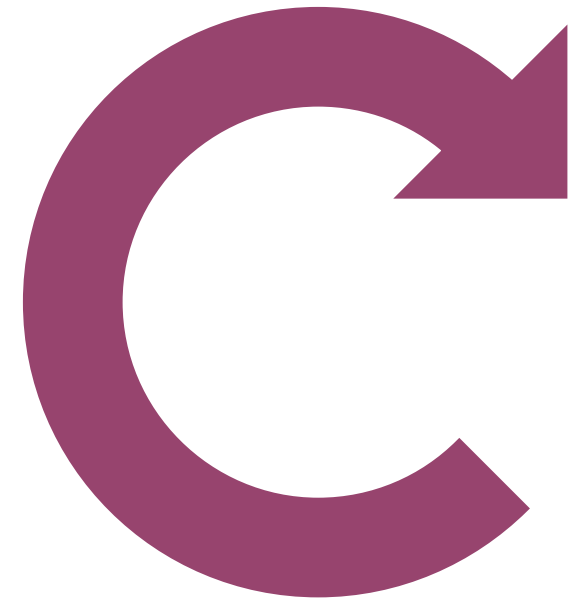
SSR renderiza páginas no servidor a cada requisição, sendo perfeito para conteúdo dinâmico e personalizado como dashboards de usuário.

Embora tenha maior latência que SSG, oferece dados sempre atualizados e melhor SEO que renderização client-side.

Incremental Static Regeneration (ISR)

```
// ISR com revalidação a cada 60 segundos
export default async function ProductPage() {
  const products = await fetch('https://api.store.com/products', {
    next: { revalidate: 60 }
  })
  return <ProductList products={products} />
}
```

- ISR combina benefícios de SSG e SSR, permitindo atualizações de conteúdo sem rebuild completo.
- Páginas são servidas estaticamente mas regeneradas em background quando necessário, ideal para e-commerce e sites de notícias.



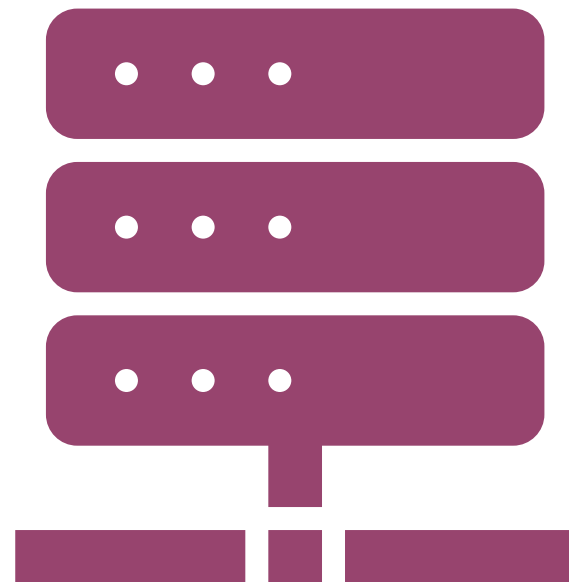
Fundamentos dos Server Components

- Server Components executam exclusivamente no servidor, oferecendo acesso direto a bases de dados, APIs backend e segredos de configuração sem exposição ao cliente.
- Reduzem significativamente o JavaScript bundle e melhoram métricas de performance como First Contentful Paint.

```
// Server Component com acesso direto ao banco  
import { db } from '@lib/database'
```

```
export default async function UsersList() {  
  const users = await db.users.findMany()
```

```
  return (  
    <div>  
      {users.map(user => (  
        <UserCard key={user.id} user={user} />  
      ))}  
    </div>  
  )  
}
```



Client Components para Interatividade

- Client Components são necessários quando a aplicação requer estado, event handlers, hooks do React ou APIs do browser.
- Devem ser marcados com 'use client' e utilizados estrategicamente para manter os benefícios dos Server Components.

'use client'

```
import { useState } from 'react'
```

```
export default function InteractiveButton() {  
  const [count, setCount] = useState(0)
```

```
  return (  
    <button onClick={() => setCount(count + 1)}>  
      Cliques: {count}  
    </button>  
  )  
}
```

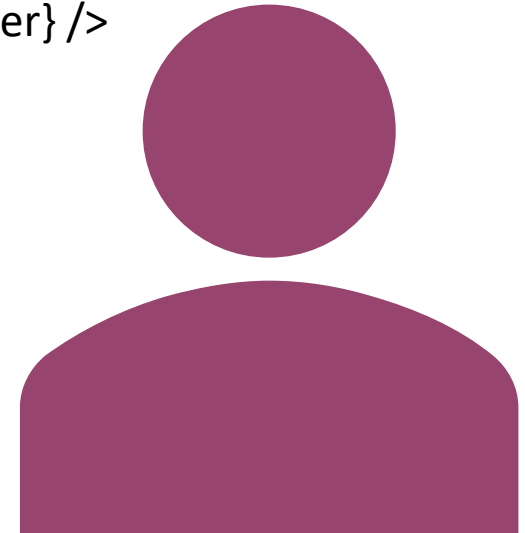


Padrões Modernos de Data Fetching

- Next.js 15 simplifica data fetching com async Server Components que utilizam a fetch API nativa com recursos de cache automático.
- A memoização automática previne requisições duplicadas e otimiza performance.

```
// Data fetching otimizado com cache
async function getUser(id: string) {
  const res = await fetch(`/api/users/${id}`, {
    cache: 'force-cache' // Cache persistente
  })
  return res.json()
}
```

```
export default async function UserProfile({ userId }: { userId: string }) {
  const user = await getUser(userId)
  return <UserDetails user={user} />
}
```



```
// API Route com validação e error handling
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'
```

```
const userSchema = z.object({
  name: z.string().min(2),
  email: z.string().email()
})
```

```
export async function POST(request: NextRequest) {
  try {
    const body = await request.json()
    const userData = userSchema.parse(body)

    const user = await createUser(userData)
    return NextResponse.json(user, { status: 201 })
  } catch (error) {
    if (error instanceof z.ZodError) {
      return NextResponse.json({ error: error.errors }, { status: 400 })
    }
    return NextResponse.json({ error: 'Internal error' }, { status: 500 })
  }
}
```

API Routes Robustas

- API Routes funcionam como backend completo dentro do Next.js, suportando todos métodos HTTP e middleware personalizado.
- A implementação deve incluir validação de entrada, tratamento de erros e respostas estruturadas.



```
// middleware.ts - Implementação responsável
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'
```

```
export function middleware(request: NextRequest) {
  const token = request.cookies.get('auth-token')?.value
  const { pathname } = request.nextUrl
```

```
  const protectedPaths = ['/dashboard', '/profile']
  const isProtected = protectedPaths.some(path => pathname.startsWith(path))
```

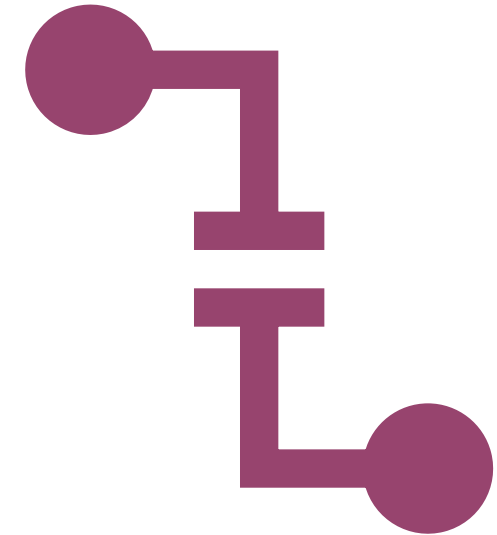
```
  if (isProtected && !token) {
    return NextResponse.redirect(new URL('/login', request.url))
  }
```

```
  return NextResponse.next()
}
```

```
export const config = {
  matcher: ['/(?!api|_next/static|_next/image|favicon.ico).*')]
}
```

Middleware e Autenticação

- Middleware em Next.js executa antes do processamento de rotas, sendo ideal para autenticação, redirects e modificações de headers.

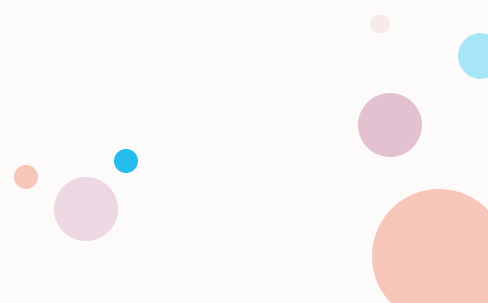


- Contudo, as recomendações de 2025 indicam cautela no uso de middleware para autenticação devido a limitações de segurança.



Data Access Layer (DAL) - Nova Abordagem Recomendada

- A abordagem moderna recomenda Data Access Layers para centralizar lógica de autenticação próxima ao acesso de dados.
- Este padrão oferece maior segurança e controle que middleware tradicional.



Otimizações Automáticas do Next.js

- Next.js oferece otimizações automáticas significativas:
 - Server Components por padrão,
 - code-splitting automático por rota,
 - prefetching de links no viewport e
 - cache inteligente de requests.
- Estas otimizações funcionam automaticamente mas podem ser configuradas para necessidades específicas.
- Image Optimization: O componente `next/image` oferece otimização automática com lazy loading, responsive images e formatos modernos.
- É essencial para manter Core Web Vitals saudáveis.

```
import Image from 'next/image'
<Image
  src="/product.jpg"
  alt="Product"
  width={600}
  height={400}
  priority={false}
  sizes="(max-width: 768px) 100vw, (max-width: 1200px) 50vw, 33vw"
/>
```

Code Splitting Dinâmico

- Utilizando next/dynamic para carregar componentes apenas quando necessário.

```
import dynamic from 'next/dynamic'
```

```
const HeavyComponent = dynamic(() => import('./HeavyComponent'), {  
  loading: () => <p>Loading...</p>,  
  ssr: false // Desabilita SSR se necessário  
})
```



Turbopack

- Nova Geração de Build

Turbopack, estável no Next.js 15, oferece melhorias dramáticas de performance: 76.7% mais rápido na inicialização do servidor local, 96.3% mais rápido em atualizações de código com fast refresh.

É escrito em Rust e otimizado especificamente para JavaScript e TypeScript.

// Metadata dinâmica

```
export async function generateMetadata({ params }: PageProps): Promise<Metadata> {  
  const post = await getPost(params.slug)
```

```
  return {  
    title: post.title,  
    description: post.description,  
    openGraph: {  
      title: post.title,  
      description: post.description,  
      images: [post.featuredImage],  
      type: 'article'  
    },  
    twitter: {  
      card: 'summary_large_image'  
    }  
  }  
}
```

SEO e Metadata

- Next.js 15 oferece sistema robusto para SEO com metadata estática e dinâmica, integração automática com Open Graph e Twitter Cards.
- O sistema é type-safe e permite configuração granular por página.



Melhores práticas para SEO



Implementação de tags semânticas HTML,



URLs canônicas,



structured data e



sitemap automático são fundamentais.



O Next.js facilita estas implementações com helpers integrados e geração automática de sitemaps.

```
// app/dashboard/error.tsx
'use client'
```

```
export default function Error({
  error,
  reset,
}: {
  error: Error & { digest?: string }
  reset: () => void
}) {
  return (
    <div>
      <h2>Algo deu errado!</h2>
      <button onClick={() => reset()}>Tentar novamente</button>
    </div>
  )
}
```

Sistema de Error Boundaries

- Next.js implementa error boundaries através de arquivos error.tsx que capturam erros em segmentos específicos da aplicação.
- Este sistema previne que erros localizados afetem toda a aplicação.

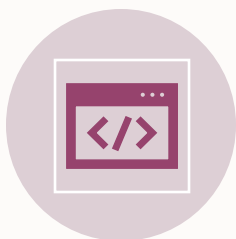


Logging e Monitoramento

Next.js 15 oferece configurações avançadas de logging para desenvolvimento e integração com serviços de monitoramento como Sentry para produção.

Logging estruturado e error reporting são essenciais para aplicações profissionais.

. Internacionalização (i18n)



Next.js oferece suporte nativo para internacionalização através de configuração automática de rotas localizadas.



A integração com bibliotecas como next-intl proporciona experiência completa de localização.



O sistema suporta sub-path routing (/pt/sobre) e domain routing (site.com.br) com detecção automática de idioma baseada em headers Accept-Language.

```
// next.config.js
module.exports = {
  i18n: {
    locales: ['pt', 'en', 'es'],
    defaultLocale: 'pt',
    domains: [
      {
        domain: 'site.com.br',
        defaultLocale: 'pt'
      }
    ]
  }
}
```

Estratégias de Teste

- Uma estratégia robusta de testes Next.js deve incluir testes unitários, integração e end-to-end.
- Jest e React Testing Library formam a base para testes unitários

```
// Teste de componente React
import { render, screen } from '@testing-library/react'
import UserProfile from './UserProfile'

test('renders user profile correctly', () => {
  const user = { name: 'João', email: 'joao@email.com' }
  render(<UserProfile user={user} />)

  expect(screen.getByText('João')).toBeInTheDocument()
  expect(screen.getByText('joao@email.com')).toBeInTheDocument()
})
```

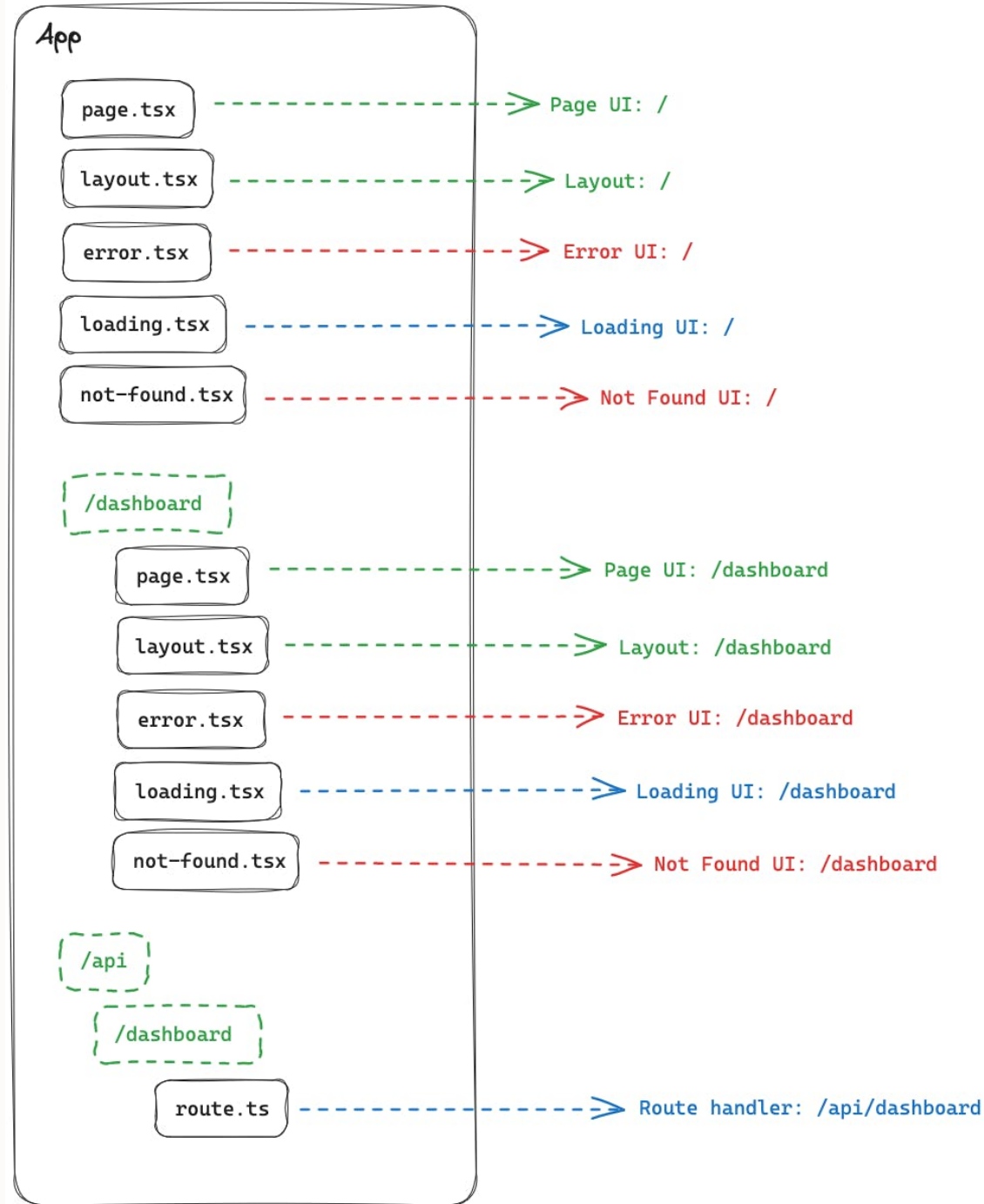
Deployment - Vercel

- A Vercel oferece deployment zero-configuration para Next.js com otimizações automáticas:
 - CDN global,
 - Incremental Static Regeneration distribuída,
 - edge functions e
 - analytics integrados.
- O processo é simplificado através de integração Git automática.



Configuração de Produção

- Preparação para produção requer:
 - otimização de build,
 - configuração de variáveis de ambiente,
 - implementação de cache headers e
 - monitoramento de performance.
- Next.js automatiza muitas otimizações mas permite configuração granular quando necessário.



Estrutura de Projeto

- Uma estrutura bem organizada segue padrões de separação por funcionalidade, com components reutilizáveis, hooks customizados, services para lógica de negócios e types para TypeScript.
- A estrutura deve escalar conforme o crescimento da aplicação.

```
src/  
├── app/           # App Router  
├── components/    # Componentes UI  
│   ├── ui/        # Componentes base  
│   ├── layout/     # Layout components  
│   └── features/    # Componentes por funcionalidade  
├── lib/           # Configurações principais  
├── hooks/         # Custom hooks  
├── services/      # Lógica de API  
├── utils/         # Funções utilitárias  
└── types/         # Tipos TypeScript
```

Convenções de Nomenclatura

- Adotar convenções consistentes facilita colaboração e manutenção:
 - PascalCase para componentes,
 - camelCase para funções e hooks,
 - kebab-case para arquivos de configuração.



Bibliotecas e Ecosistema Recomendado

- Dependências Essenciais:
 - Autenticação: NextAuth.js ou Auth0 para implementação robusta de autenticação
 - Validação: Zod para validação type-safe de schemas
 - Estado: Zustand ou React Query para gerenciamento de estado
 - UI: Tailwind CSS + Shadcn/ui ou Chakra UI para interfaces consistentes
 - Database: Prisma como ORM type-safe

```
{  
  "dependencies": {  
    "next": "^15.0.0",  
    "react": "^18.0.0",  
    "react-dom": "^18.0.0",  
    "zod": "^3.22.0",  
    "prisma": "^5.0.0",  
    "@prisma/client": "^5.0.0",  
    "tailwindcss": "^3.3.0"  
  }  
}
```

Implementação

```
npx create-next-app@latest .
```

```
npm run dev
```

- Configuração TypeScript e ESLint
 - Next.js configura automaticamente TypeScript e ESLint com configurações otimizadas.
- Setup do Tailwind CSS

```
npm install tailwindcss postcss autoprefixer
```

```
npx tailwindcss init -p
```

Desenvolvimento de Funcionalidades



Server Component para Data Fetching

Implementar busca de dados no servidor com cache otimizado.



Client Components para Interatividade

Adicionar componentes interativos marcados com 'use client'.



API Routes para Backend

Criar endpoints robustos com validação e error handling.



Middleware para Autenticação

Implementar proteção de rotas com middleware responsável.

Configuração de Produção

- Otimizações de Build

```
// next.config.js
const nextConfig = {
  images: {
    domains: ['cdn.example.com'],
    formats: ['image/webp', 'image/avif']
  },
  compress: true,
  poweredByHeader: false
}
```